

Base de Datos (75.15 / 95.05 / TA044)

Segundo Parcial Promocional - 25 de junio de 2025 - 2025141

Costos		Costos		NoSQL		Padrón: Apellido: Nombre: Hojas entregadas:
NoSQL		Conc.		Recup.		
Nota: ☐ Aprobado ☐ Insuficiente						

Criterio de aprobación: El examen está compuesto por 6 ítems, cada uno de los cuales se corrige como B/B-/Reg/Reg-/M. El examen se aprueba con nota mayor o igual a 4 (cuatro) y la condición de aprobación es desarrollar un ítem bien (B/B-) en los 3 grupos de ejercicios (costos, NoSQL, concurrencia/recuperación). Adicionalmente, no deberá haber más de dos ítems mal o no desarrollados.

1. (*Costos*) El famoso banco “*Mc Bancon*” desea “hacer Data Analytics” (sic) con los préstamos de sus clientes:

- **Clientes** (id_cliente, nombre, provincia, tipo,)
- **Préstamos** (id_prestamo, fecha, monto, id_cliente,

Se busca analizar los préstamos recientes de clientes premium de Catamarca:

```
SELECT c.id_cliente, p.fecha, p.monto
FROM clientes c INNER JOIN prestamos p USING(id_cliente)
WHERE c.provincia = 'Catamarca' AND c.tipo = 'Premium'
AND p.fecha >= '2024-01-01'
```

Estime el **costo** de realizar esta consulta y la **cantidad de filas** que serán devueltas, contando la siguiente información de catálogo, sabiendo que se dispone de 100 bloques de memoria disponibles para la operación, que el único índice de clustering es el de fecha en la tabla préstamos (los otros no son de clustering) y que un 5% de los préstamos son de 2024 o posteriores.

CLIENTES	PRESTAMOS
n(clientes) = 100.000	n(prestamos) = 100.000
B(clientes) = 20.000	B(prestamos) = 10.000
V(provincia, clientes) = 20	V(id_cliente, prestamos) = 10.000
V(tipo, clientes) = 4	V(fecha, prestamos) = 2.000
H(I(provincia, clientes)) = 2	H(I(id_cliente, prestamos)) = 3
H(I(tipo, clientes)) = 1	H(I(fecha, prestamos)) = 4

2. (Costos) Adalberto está preparando un gran festejo y busca un lugar donde hacerlo, para eso revisa la plataforma Mooglee Reviews revisando puntuaciones de sus dos influencers favoritos: “Fulki” y “D.J. Ann”. Ejecuta entonces la siguiente consulta:

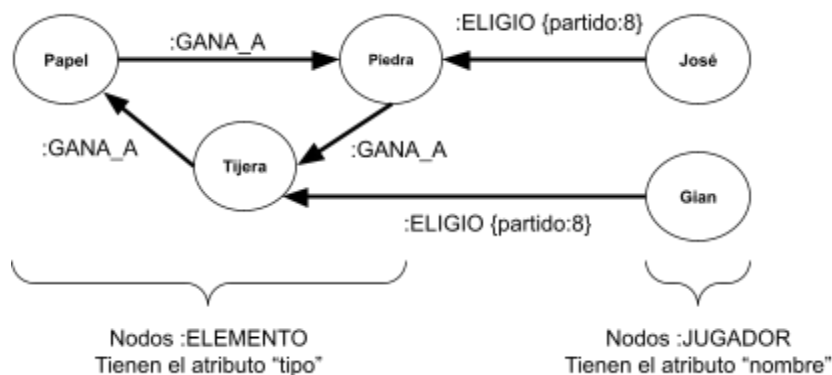
```
SELECT * FROM lugares l INNER JOIN puntuaciones p USING (id_lugar)
WHERE (p.usuario = 'Fulki' OR p.usuario = 'D.J. Ann') AND p.puntaje = 10
```

- Puntuaciones (usuario, id_lugar, fecha, puntaje)
- Lugares (id_lugar, nombre, direccion, precio_promedio)

Estime la cantidad de filas y bloques devueltos por la consulta, sabiendo que los influencers solamente ponen un 10 en un 5% de sus puntuaciones. Para mejorar la estimación se sabe que la tabla de puntuaciones tiene un histograma que registra los 3 usuarios con más puntuaciones: “Bey Roo” con 100 mil puntuaciones, “Fulki” con 70 mil y “Cebollín” con 50 mil.

PUNTUACIONES	LUGARES
$n(\text{Puntuaciones}) = 1.000.000$	$n(\text{Lugares}) = 50.000$
$B(\text{Puntuaciones}) = 100.000$	$B(\text{Lugares}) = 10.000$
$V(\text{usuario}, \text{Puntuaciones}) = 100.000$	

3. (NoSQL) La AAPPT (Asociación Argentina de Piedra Papel o Tijera) guarda en una base de datos la elección que hizo cada jugador en un partido, pudiendo entonces saber quién ganó debido a que cada tipo de elemento gana contra uno y pierde contra otro. Por ejemplo, la siguiente estructura indica que José le ganó a Gian en el partido 8 :



Para ayudar a una investigación de fraude, escriba una consulta en Cypher que devuelva el nombre de los jugadores que cumplan las siguientes 3 condiciones:

- Siempre hayan perdido contra el jugador de nombre “Jules”, habiéndolo enfrentado al menos una vez.
- Hayan ganado en total más de 100 partidos.
- Hayan perdido en total entre 1 y 10 partidos.

4. (NoSQL) La facultad guarda las notas finales de los alumnos en una base MongoDB en la que cada documento representa un cuatrimestre de una materia y sus notas:

```
{
  año: 2025, cuatrimestre: 1,
  materia: "TA044 - Base de datos", departamento: "Computación",
  notas: [
    { padron: 12345, nombre: "Sawa Mielnik", notaFinal: 7 },
    { padron: 12543, nombre: "Matias Wester", notaFinal: 2 },
    { padron: 12999, nombre: "Marcelo Giovanni", notaFinal: 10 },
    (...) ]
}
```

Para evaluar a los departamentos, la comitiva de burocracia quiere saber, para cada departamento de la facultad, cuántas materias hay en ese departamento con un promedio de nota de aprobación mayor a 8. Haga una consulta que devuelva esos datos (se adjunta respuesta de ejemplo).

Importante: Considere que se aprueba la materia con nota mayor o igual a 4 y no hay que devolver departamentos para los que no haya materias que cumplan la condición (es decir, con cantidad 0 de materias).

```
{departamento: "Computación", cant_mate_mayor_8: 3 },
{departamento: "Electrónica", cant_mate_mayor_8: 1 }, (....)
```

5. (Concurrencia y Transacciones) Dado el siguiente solapamiento de transacciones:

$b_{T1}; b_{T2}; R_{T1}(X); R_{T2}(X); b_{T3}; W_{T3}(Y); W_{T3}(Z); W_{T1}(Y); W_{T2}(X); R_{T2}(Z);$

- Indique, con una justificación mínima, si el solapamiento es serializable. En caso de serlo, indique un orden de transacciones equivalente.
- Agregue, luego de las operaciones, los commits de todas las transacciones para que el solapamiento sea recuperable. En caso de no ser posible, justificarlo.
- Indique si el solapamiento **permite o evita** rollbacks en cascada.

6. (Recuperación) Un SGBD implementa el algoritmo de recuperación UNDO con checkpoint activo. Luego de una falla, el sistema encuentra el siguiente archivo de log (a la derecha):

Explique **cómo se llevará a cabo** el procedimiento de recuperación, indicando **hasta qué punto del archivo** de log se deberá retroceder, y **qué cambios** deberán ser realizados en disco y en el archivo de log.

```
01 (BEGIN, T1);
02 (WRITE T1, A, 1);
03 (WRITE T1, B, 2);
04 (BEGIN, T2);
05 (COMMIT, T1);
06 (WRITE T2, B, 3);
07 (BEGIN, T3);
08 (BEGIN CKPT, T2, T3);
09 (WRITE T3, C, 4);
10 (COMMIT, T2)
11 (BEGIN, T4);
12 (WRITE T4, D, 5);
13 (COMMIT, T4);
```